UNIVERSIDAD NACIONAL AUTÓNOMA DE NICARAGUA, LEÓN

ÁREA DE CONOCIMIENTO DE CIENCIAS Y TECNOLOGÍA
INGENIERÍA EN SISTEMAS DE INFORMACIÓN

BODETIME

Sistema para el Control de Horarios de Entrada y Salida de Trabajadores

Proyecto Integrador II
Integrantes: Leonardo de Jesús Pichardo Baltodano y Ángel de Jesús Muñoz
Carrera: Ingeniería en Sistemas de Información
Año: Segundo | Grupo: 2
Tutor: Edliberto Lacayo
León, Nicaragua • 2026

Resumen ejecutivo

BODETIME es una aplicación de escritorio para administrar trabajadores y registrar su asistencia. Permite crear, consultar, editar y desactivar trabajadores; registrar entradas y salidas; revisar tardanzas con un horario configurable; consultar el historial y generar reportes PDF. La interfaz está hecha con PySide6. Los servicios aplican las reglas, SQLAlchemy administra las sesiones y MySQL conserva los datos. ReportLab arma los PDF con filas obtenidas desde la base. La revisión local incluyó ocho pruebas unitarias y una verificación temporal de integración con MySQL.

1. Introducción

En una bodega, el control de asistencia permite conocer quién se presentó a trabajar y a qué hora realizó su marcaje. Cuando esta información se lleva únicamente en registros manuales, pueden presentarse errores de transcripción, pérdida de hojas, consultas lentas y dificultades para revisar el historial. BODETIME plantea digitalizar ese proceso mediante una aplicación de escritorio que almacene los datos de manera estructurada y permita consultarlos con rapidez.

La aplicación se diseña para que el usuario registre trabajadores, marque entradas y salidas, consulte los registros y genere reportes. La hora del marcaje debe obtenerse desde el reloj del sistema en el momento de la operación; el horario laboral y la tolerancia de tardanza deben ser configurables por el responsable, no valores supuestos en el informe.

2. Planteamiento del problema

Problema central: falta de un mecanismo digital centralizado y verificable para registrar y consultar los horarios de entrada y salida de los trabajadores de una bodega.

2.1 Pregunta del problema

¿Cómo puede una aplicación de escritorio facilitar el registro, consulta y seguimiento de la asistencia de los trabajadores, reduciendo errores asociados al control manual?

2.2 Causas y efectos (Ishikawa)

Categoría	Causas posibles	Efecto en el proceso
Método	Registro manual; ausencia de un procedimiento uniforme.	Diferencias entre registros y dificultad para verificar horarios.
Personas	Dependencia de quien anota; errores al escribir nombres u horas.	Datos incompletos o inconsistentes.
Tecnología	No existe una herramienta centralizada o respaldos sistemáticos.	Pérdida de información y consultas lentas.
Información	Formatos distintos y falta de historial organizado.	Dificultad para elaborar reportes y detectar tardanzas.
Control	Horario y criterios de tardanza no documentados.	Evaluaciones no uniformes de puntualidad.

El diagrama de Ishikawa organiza hipótesis de causas; no demuestra por sí solo que todas ocurran en la bodega. Deben validarse mediante observación o entrevista con el usuario del proceso.

3. Objetivos

3.1 Objetivo general

Desarrollar una aplicación de escritorio que permita gestionar trabajadores y controlar sus registros de asistencia mediante operaciones CRUD, almacenamiento en MySQL y generación de reportes PDF.

3.2 Objetivos específicos

- Registrar, consultar, actualizar y eliminar la información de los trabajadores.
- Registrar las horas de entrada y salida asociadas a cada trabajador.
- Consultar los marcajes por trabajador y por rango de fechas.
- Identificar tardanzas comparando el marcaje con un horario configurable.
- Generar reportes PDF desde la interfaz gráfica con datos recuperados de MySQL.
- Validar los datos ingresados y comunicar errores o confirmaciones mediante cuadros de diálogo.
- Aplicar pruebas funcionales para comprobar la persistencia y consistencia de la información.

4. Justificación

El sistema reúne los datos de asistencia en un solo lugar. Así se puede consultar el historial y preparar reportes con un criterio uniforme. El proyecto integra una interfaz de escritorio, reglas de negocio, una base de datos relacional y generación de documentos PDF.

5. Alcance y delimitaciones

5.1 Funciones implementadas

- Mantenimiento de trabajadores: alta, consulta, modificación y baja lógica o eliminación controlada.
- Registro de asistencia: entrada y salida con fecha y hora generadas automáticamente.
- Consulta de asistencia por fecha, trabajador y estado.
- Cálculo de tardanza con base en hora de inicio y tolerancia configurables.
- Reporte PDF de asistencia con filtros y fecha de generación.
- Validaciones de campos obligatorios, formatos, duplicados y secuencia entrada/salida.

5.2 Fuera del alcance de esta versión

- Relojes biométricos, tarjetas RFID o integración con dispositivos externos.
- Cálculo de nómina, deducciones salariales o gestión de vacaciones.
- Acceso remoto multi-sucursal y sincronización en la nube.
- Aplicación móvil y notificaciones automáticas.

Estas exclusiones evitan prometer funciones que no se desprenden de la rúbrica ni del alcance inicial.

6. Marco teórico

6.1 Sistema de información

Conjunto organizado de recursos que captura, procesa, almacena y presenta información para apoyar actividades y decisiones. BODETIME transforma marcajes y datos del personal en registros consultables.

6.2 Control de asistencia

Proceso de registrar la presencia de las personas y los momentos de entrada y salida. La calidad del control depende de reglas claras, datos íntegros y trazabilidad.

6.3 Base de datos relacional

Modelo que organiza información en tablas relacionadas mediante claves primarias y foráneas. La relación entre trabajadores y asistencias permite conservar el historial de cada persona.

6.4 ORM y SQLAlchemy

Un mapeador objeto-relacional vincula clases de Python con tablas de la base de datos. SQLAlchemy permite definir modelos, administrar sesiones y ejecutar operaciones persistentes sin construir manualmente cada sentencia SQL.

6.5 Interfaz gráfica con PySide6

PySide6 ofrece bindings de Qt para Python. Sus widgets, señales y slots permiten construir formularios, tablas, botones y cuadros de diálogo para la interacción con el usuario.

6.6 Generación de PDF con ReportLab

ReportLab permite construir documentos PDF en Python. Su módulo Platypus organiza contenido mediante elementos como párrafos y tablas; TableStyle controla encabezados, bordes, alineación y espaciado.

6.7 Validación y transacciones

La validación comprueba datos antes de guardarlos. Las transacciones agrupan operaciones para confirmar todos los cambios o revertirlos cuando ocurre un error, evitando estados parciales.

6.8 Scrum

Scrum es un marco de trabajo para desarrollar productos en incrementos. El equipo mantiene un Product Backlog, selecciona trabajo para cada Sprint, revisa el incremento y analiza oportunidades de mejora.

7. Metodología de desarrollo: Scrum

El equipo organizó el trabajo de forma incremental y por módulos. Los dos integrantes participaron en el desarrollo y las pruebas; los roles de Scrum ayudaron a repartir responsabilidades.

Integrante	Responsabilidad Scrum	Actividades principales
Ángel Muñoz	Product Owner y Developer	Representar necesidades, priorizar el backlog, revisar criterios de aceptación y desarrollar funcionalidades acordadas.
Leonardo Pichardo	Scrum Master y Developer	Facilitar la organización del trabajo, dar seguimiento a obstáculos, apoyar la calidad e implementar funcionalidades.

7.1 Eventos

Evento	Propósito	Evidencia sugerida
Sprint Planning	Definir objetivo y seleccionar tareas del Sprint.	Objetivo, tareas, responsables y estimación.
Daily Scrum	Inspeccionar avance y obstáculos.	Actualización breve del tablero.
Sprint Review	Mostrar el incremento funcional y recibir comentarios.	Demostración de la aplicación.
Sprint Retrospective	Identificar qué mejorar en el siguiente Sprint.	Acuerdos de mejora.

7.2 Product Backlog priorizado

ID	Historia / funcionalidad	Prioridad	Criterio de aceptación resumido
PB-01	Gestionar trabajadores (CRUD).	Alta	Se crea, consulta, actualiza y elimina/desactiva un trabajador en MySQL.
PB-02	Registrar entrada.	Alta	Guarda trabajador, fecha y hora reales; evita marcajes inválidos.
PB-03	Registrar salida.	Alta	Completa el marcaje abierto del trabajador; no admite salida sin entrada.
PB-04	Consultar asistencia.	Alta	Filtra y muestra datos persistidos por fecha y trabajador.
PB-05	Calcular tardanza.	Alta	Compara hora de entrada con horario y tolerancia configurados.
PB-06	Generar PDF desde GUI.	Alta	El botón genera un PDF con los resultados del filtro actual.
PB-07	Validar entradas y mostrar mensajes.	Alta	Los errores se explican con QMessageBox sin cerrar inesperadamente la app.
PB-08	Administrar usuarios y permisos.	Media	Acceso protegido por roles, si se implementa dentro del alcance acordado.

7.3 Sprint backlog sugerido

Sprint	Objetivo	Tareas verificables
Sprint 1	Base técnica y trabajadores	Crear esquema MySQL; configurar SQLAlchemy; implementar modelo Trabajador; CRUD; validaciones.
Sprint 2	Marcajes y consultas	Modelo Asistencia; entrada/salida; reglas de duplicidad; filtros; pruebas de integridad.
Sprint 3	Reportes y cierre	Reporte PDF con ReportLab; integración al botón; pruebas completas; corrección de interfaz; preparación de defensa.

Las fechas del primer Sprint corresponden al plan inicial. El estado de las funciones que se comprobaron se resume en la sección Resultados y discusión.

8. Requisitos del sistema

8.1 Requisitos funcionales

ID	Requisito
RF-01	El sistema debe permitir registrar trabajadores con nombre y datos básicos definidos.
RF-02	El sistema debe mostrar la lista de trabajadores almacenados en MySQL.
RF-03	El sistema debe permitir actualizar la información de un trabajador.
RF-04	El sistema debe permitir eliminar o desactivar un trabajador con confirmación.

ID	Requisito
RF-05	El sistema debe registrar la hora de entrada usando fecha y hora del sistema.
RF-06	El sistema debe registrar la hora de salida vinculada a una entrada abierta.
RF-07	El sistema debe consultar asistencia por trabajador y rango de fechas.
RF-08	El sistema debe identificar tardanzas según horario y tolerancia configurables.
RF-09	El sistema debe generar un PDF desde un botón de la interfaz gráfica.
RF-10	El sistema debe mostrar mensajes claros ante datos inválidos y operaciones exitosas.

8.2 Requisitos no funcionales

ID	Requisito
RNF-01	La interfaz debe ser consistente, legible y fácil de navegar.
RNF-02	La persistencia debe realizarse en MySQL mediante SQLAlchemy ORM.
RNF-03	Las operaciones de escritura deben utilizar transacciones y manejo de errores.
RNF-04	Las credenciales de conexión no deben quedar escritas directamente en el código fuente.
RNF-05	Los campos y relaciones deben proteger la integridad de los datos.
RNF-06	La aplicación debe informar si no puede conectarse a la base de datos.
RNF-07	El código debe estar separado por responsabilidades para facilitar mantenimiento.
RNF-08	El reporte debe mostrar fecha de generación, filtros aplicados y encabezados legibles.

9. Diseño técnico de la solución

9.1 Arquitectura por capas

Capa	Tecnología	Responsabilidad
Presentación	PySide6	Ventanas, formularios, tablas, navegación y QMessageBox.
Aplicación / servicios	Python	Reglas de asistencia, validaciones, coordinación de operaciones y filtros.
Persistencia	SQLAlchemy ORM	Modelos, sesiones, transacciones y consultas.
Datos	MySQL	Almacenamiento persistente, claves, restricciones e índices.
Reportes	ReportLab	Construcción del PDF con datos consultados desde la capa de servicios.

Flujo: Usuario → PySide6 → Servicio de aplicación → SQLAlchemy Session → MySQL. Para reportes: PySide6 → Servicio de reportes → consulta de datos → ReportLab → archivo PDF.

9.2 Modelo relacional

Tabla	Campos principales	Reglas
trabajadores	id_trabajador (PK), codigo (UNIQUE), nombres, apellidos, cargo, telefono, activo, creado_en	codigo único; nombres y apellidos obligatorios.
asistencias	id_asistencia (PK), id_trabajador (FK), fecha, hora_entrada, hora_salida, tarde, observacion	Una asistencia por trabajador y fecha; la salida debe ser posterior a la entrada.
configuracion_horario	id_configuracion (PK), hora_inicio, tolerancia_minutos, actualizado_en	Guarda la hora de inicio y la tolerancia usada para evaluar tardanzas.

Un trabajador puede tener varias asistencias a lo largo del tiempo. Cada asistencia pertenece a un trabajador. La restricción UNIQUE(id_trabajador, fecha) limita la versión actual a un registro diario por persona.

9.3 Reglas de negocio

- No se registra asistencia para un trabajador inactivo.
- La entrada se genera con la fecha y hora del sistema; el usuario no debe poder alterar el marcaje ordinario.
- No se permite registrar una segunda entrada diaria abierta para el mismo trabajador.
- La salida solo se registra si existe una entrada sin salida.
- La hora de salida debe ser posterior a la hora de entrada.
- La tardanza se calcula comparando la hora de entrada con la hora de inicio más la tolerancia definida.
- El trabajador se desactiva para que deje de aparecer como opción de marcaje y se conserve su historial.

9.4 Tecnologías

Tecnología	Uso en BODETIME
Python	Lenguaje principal para lógica y reglas.
PySide6	Interfaz gráfica de escritorio basada en Qt.
MySQL	Base de datos relacional exigida para la demostración.
SQLAlchemy ORM	Mapeo de clases a tablas y operaciones CRUD.
PyMySQL	Driver DBAPI para conectar SQLAlchemy con MySQL.
ReportLab	Generación del PDF desde la aplicación.
Visual Studio Code	Edición y depuración del código.
Git	Control de versiones y trazabilidad de cambios.
Scrum	Organización incremental del trabajo.

10. Diseño de interfaz y navegación

La ventana principal organiza cinco vistas: Inicio, Trabajadores, Asistencia, Reportes y Configuración. Las tablas se actualizan después de cada operación y los cuadros de diálogo informan errores o confirmaciones.

Vista	Elementos mínimos	Validaciones / mensajes
Inicio	Resumen de trabajadores activos, marcajes del día y tardanzas.	Aviso visible si la base de datos no está disponible.
Trabajadores	Formulario y tabla con botones Nuevo, Guardar, Editar, Eliminar y Limpiar.	Campos requeridos, código duplicado y confirmación antes de eliminar.
Asistencia	Selector de trabajador, botones Entrada/Salida y tabla de registros.	Evitar duplicados, salida sin entrada y salida anterior a entrada.
Reportes	Filtros por fecha/trabajador y botón Generar PDF.	Aviso si no hay datos; confirmar ruta o mostrar dónde se guardó el archivo.
Configuración	Hora de inicio y tolerancia.	Validar formato de hora y valores no negativos.

11. Plan de pruebas y criterios de aceptación

La tabla resume acciones útiles para la demostración. La sección Resultados y discusión separa lo que se comprobó en la integración local de las acciones que conviene ensayar desde la ventana antes de exponer.

ID	Prueba / acción	Resultado esperado	Evidencia
T-01	Crear trabajador con datos válidos.	Se guarda en MySQL y aparece en la tabla.	Captura GUI y fila consultada.
T-02	Consultar trabajador existente.	Se muestran sus datos actuales desde MySQL.	Captura de tabla.
T-03	Editar un dato y guardar.	El cambio persiste al recargar la vista.	Antes/después.
T-04	Eliminar/desactivar trabajador.	Pide confirmación y aplica la regla definida.	Mensaje y consulta posterior.
T-05	Guardar trabajador con campos vacíos.	Bloquea operación y muestra QMessageBox.	Captura del mensaje.
T-06	Registrar entrada.	Guarda fecha y hora y actualiza la tabla.	Marcaje en GUI y BD.
T-07	Registrar entrada duplicada.	Rechaza el marcaje según regla diaria.	Mensaje de validación.
T-08	Registrar salida sin entrada.	Rechaza la operación.	Mensaje de validación.
T-09	Registrar salida válida.	Completa el registro y persiste en MySQL.	Fila con entrada/salida.
T-10	Generar reporte desde botón.	Crea PDF con datos reales y encabezados.	PDF generado y abierto.
T-11	Filtrar rango sin resultados.	Muestra aviso y no genera un reporte engañoso.	Mensaje.
T-12	Desconectar MySQL y abrir módulo.	La aplicación informa el problema sin cerrarse.	Mensaje de conexión.

12. Resultados y discusión

La ventana de BODETIME abrió con MySQL conectado. En una verificación temporal de integración, el servicio completó alta, consulta, actualización y desactivación lógica de un trabajador; rechazó un código duplicado, una entrada diaria duplicada y una salida anterior a la entrada; registró entrada y salida válidas; consultó el historial; y generó un PDF con filas recuperadas de MySQL. Los registros temporales se retiraron y la configuración de horario anterior se restauró. Ocho pruebas unitarias también pasaron.

Indicador	Resultado observado	Evidencia a adjuntar
CRUD de trabajadores	Alta, consulta, actualización y desactivación lógica verificadas en una integración temporal con MySQL.	Los registros de prueba se retiraron al finalizar.
Persistencia MySQL	La conexión SQLAlchemy/PyMySQL respondió y la ventana mostró MySQL conectado.	Consulta de conexión y panel principal.
Marcajes entrada/salida	Se comprobaron entrada, salida, tardanza, rechazo de duplicado, salida inválida e historial.	Prueba de integración con datos temporales.
Validaciones visuales y dominio	Ocho pruebas unitarias aprobadas; la aplicación informa errores de conexión y mantiene abierta la ventana.	pytest -q, 3 de noviembre de 2026.

Indicador	Resultado observado	Evidencia a adjuntar
Generación PDF	ReportLab creó un PDF usando filas leídas de MySQL. El diálogo de guardado no se verificó manualmente desde la GUI.	PDF temporal eliminado al terminar.

La interfaz delega las operaciones a los servicios. Los servicios validan cada solicitud y usan sesiones SQLAlchemy para consultar o guardar en MySQL. El generador ReportLab recibe las filas consultadas y crea el PDF. Esta división permite revisar cada parte por separado. El diálogo de guardado del botón de PDF debe ensayarse manualmente antes de presentarlo al jurado.

13. Conclusiones

- BODETIME reúne la gestión de trabajadores y el control diario de asistencia en una aplicación de escritorio.
- PySide6 muestra las pantallas; los servicios aplican reglas; SQLAlchemy y MySQL guardan y consultan la información.
- La aplicación rechaza marcajes duplicados y salidas sin entrada, y calcula tardanzas con el horario configurado.
- ReportLab arma reportes PDF con resultados consultados en MySQL.
- Las verificaciones locales cubrieron el servicio y la base. La demostración ante el jurado debe mostrar las acciones desde la GUI.

14. Recomendaciones

- Antes de exponer, abre la aplicación y confirma que el indicador de MySQL aparece conectado.
- Configura la hora de entrada y la tolerancia antes de probar marcajes.
- Usa un trabajador de prueba autorizado y no muestres datos personales en las diapositivas.
- Mantén las credenciales solo en el archivo .env local y fuera del repositorio.
- Ensaya el botón Generar PDF y confirma que el archivo se abre desde la ventana.
- Explica que esta versión permite un turno por persona y fecha; no incluye biometría ni nómina.
- Confirma que la portada incluye a Edliberto Lacayo como tutor.

15. Guion técnico para la demostración

Orden	Acción	Qué explicar al jurado
1	Abrir Inicio y revisar el estado de MySQL.	La pantalla resume trabajadores, asistencia y tardanzas consultando la base.
2	Abrir Trabajadores y crear un registro de prueba autorizado.	El formulario valida los campos y el servicio guarda el trabajador con SQLAlchemy.
3	Consultar, editar y desactivar el registro de prueba.	Estas acciones corresponden a las operaciones CRUD; la baja conserva el historial.
4	Abrir Configuración y revisar el horario y la tolerancia.	La tardanza depende de esos valores guardados en MySQL.
5	Marcar entrada y salida; intentar repetir una entrada.	Se muestra el registro guardado y un QMessageBox al rechazar el duplicado.
6	Filtrar el historial y generar el PDF desde Reportes.	El servicio consulta MySQL y ReportLab arma la tabla del informe.

Orden	Acción	Qué explicar al jurado
7	Cerrar con una regla y un límite del sistema.	La salida requiere una entrada; la versión actual admite un turno al día.

16. Preguntas probables del jurado

¿Por qué MySQL y no SQLite?
La rúbrica pide que la aplicación demuestre persistencia real con MySQL. El programa se conecta con PyMySQL y SQLAlchemy.

¿Qué es SQLAlchemy ORM?
Permite trabajar con objetos de Python que representan filas de las tablas. La sesión guarda o consulta esos objetos.

¿Qué significa CRUD?
Significa crear, consultar, actualizar y eliminar. En BODETIME, eliminar un trabajador es desactivarlo para conservar sus asistencias.

¿Cómo se calcula una tardanza?
Se compara la entrada con la hora de inicio más la tolerancia. Si la entrada supera ese límite, se marca tarde.

¿Cómo se evita una salida sin entrada?
El servicio busca una entrada abierta de ese día. Si no existe, rechaza la salida.

¿Cómo se genera el PDF?
El servicio recupera las filas de MySQL y ReportLab las coloca en una tabla con encabezados y formato.

¿Qué función tiene Scrum?
Ayuda a dividir el trabajo en partes pequeñas, revisar avances y ajustar las siguientes tareas.

¿Por qué se desactiva un trabajador en lugar de borrarlo?
La desactivación evita que la persona vuelva a marcar, pero conserva sus asistencias anteriores.

¿La aplicación permite varios turnos al día?
No en esta versión. La base limita a un registro por trabajador y fecha.

17. Referencias técnicas

- SQLAlchemy. MySQL and MariaDB — SQLAlchemy 2.0 Documentation. https://docs.sqlalchemy.org/en/20/dialects/mysql.html
- SQLAlchemy. ORM Quick Start. https://docs.sqlalchemy.org/en/20/orm/quickstart.html
- Qt for Python. PySide6 Documentation. https://doc.qt.io/qtforpython-6/
- Qt for Python. QMessageBox. https://doc.qt.io/qtforpython-6/PySide6/QtWidgets/QMessageBox.html
- ReportLab. Chapter 7: Tables and TableStyles. https://docs.reportlab.com/reportlab/userguide/ch7_tables/
- ReportLab. ReportLab User Guide. https://docs.reportlab.com/reportlab/userguide/
- Scrum Guides. The Scrum Guide. https://scrumguides.org/scrum-guide.html

Las tecnologías y reglas descritas corresponden a la implementación de BODETIME y a los requisitos de la rúbrica.